CMPEN/EE 454: Project 2 Write-Up

Group 1: Cole Kvasnak, Kyle Nicarry, Ben Rishel, Gavin Sweeney

Results for Task 3.1

How we solved it:

This question was pretty straightforward. For the initial questions regarding which parameters were internal and external, we just consulted the lecture notes from class. Any parameters that are specific to the camera's internal characteristics are considered internal parameters. For the next two sections, we also used lecture notes. To aid in our understanding of the construction of Kmat and Pmat, we used our MATLAB ability to construct the matrices from scratch. This allowed us to confirm that we were using the correct parameters. Both of these portions of MATLAB code were simple matrix construction and then difference calculations to check the accuracy of our construction.

Which are the internal parameters?

The internal parameters are the parameters that are specific to the camera. In the parameter files, this includes foclen (focal length), prinpoint (principal point), aspectratio (ratio of the y focal length to the x focal length), skew, and radial.

Which are the external parameters?

The external parameters are the position of the camera center in world coordinates, the orientation of the camera's rotation, and Rmat which is the 3x3 rotation matrix equivalent to the orientation parameter which is a unit quaternion.

Which internal parameters combine to form the matrix Kmat?

Kmat is the multiplication of the film plane to pixels 3x3 matrix and the perspective projection matrix, which yields a 3x4 matrix which can be simplified into a 3x3 matrix times a 3x4 matrix [I | 0]. Its entries are determined by the internal parameters: foclen, prinpoint, aspectratio, and skew. The foclen seems to already be in pixel units, so that is why no mm to pixel conversion is needed. Here, you can see the definition of the Kmat matrix:

$$\begin{bmatrix} \pm f/s_x & 0 & o_x \\ 0 & \pm f/s_y & o_y \\ 0 & 0 & 1 \end{bmatrix}$$

where f is the focal length, s_x and s_y are the mm to pixel conversion combined with the aspect ratio (square pixels in this instance), and o_x and o_y are the principal points of the image. Skew would be in row 1, column 2, but in this instance, the skew parameter is equal to zero.

In task3_1.m, we tested the construction of this matrix from scratch using the process outlined above. Then, we tested the max absolute difference between parameters:

```
>> task3_1
Max absolute difference between test Kmat and provided Kmat (camera 1): 0
Max absolute difference between test Kmat and provided Kmat (camera 2): 0
```

This verifies our approach.

Which external parameters combine to form the matrix Pmat?
Pmat is the combination of the world to camera matrices (the 4x4 rotation matrix and the 4x4 translation matrix). This uses the external parameters: the position of the camera center in world coordinates (position) and the orientation of the camera's rotation (orientation). The orientation is converted from a unit quaternion into Rmat which is the 3x3 rotation matrix used in Pmat. Pmat is defined as:

$$\begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{pmatrix}$$

We test this in task3_1.m by manually constructing Pmat for both camera 1 and camera 2. The main computation here is calculating t_x, t_y, and t_z which are simply a negation and rotation of the position of the camera's center in world coordinates. The rest of the matrix is given in Rmat. Calculating the max absolute difference between our constructed matrix and the given Pmat verifies our construction and understanding:

```
Max absolute difference between test Pmat and provided Pmat (camera 1): 9.09495e-13
Max absolute difference between test Pmat and provided Pmat (camera 2): 1.13687e-13
```

What is the location of the camera?
The location of the camera in world coordinates is given by the position matrix. These are given in millimeters. This can be easily printed in MATLAB:

```
Camera 1 location (mm): [-4479.6, 5524.7, 1899.0]
Camera 2 location (mm): [4416.9, 5513.8, 1788.9]
```

Verify that location of the camera and the rotation Rmat of the camera combine in the expected way to yield the appropriate entries in Pmat.
This was verified in the "Which external parameters combine to form the matrix Pmat?" section.

Any implementation decisions we made that were clever or unusual:
This question was fairly straightforward in terms of implementation, but one clever decision we made was when constructing Pmat, we used matrix multiplication to acquire [tx, ty, tz] rather than computing them individually and then entering them into a matrix with R manually as well. This allowed us to combine two partitioned matrices rather than constructing an entirely new one from scratch with computation at each position in the matrix.

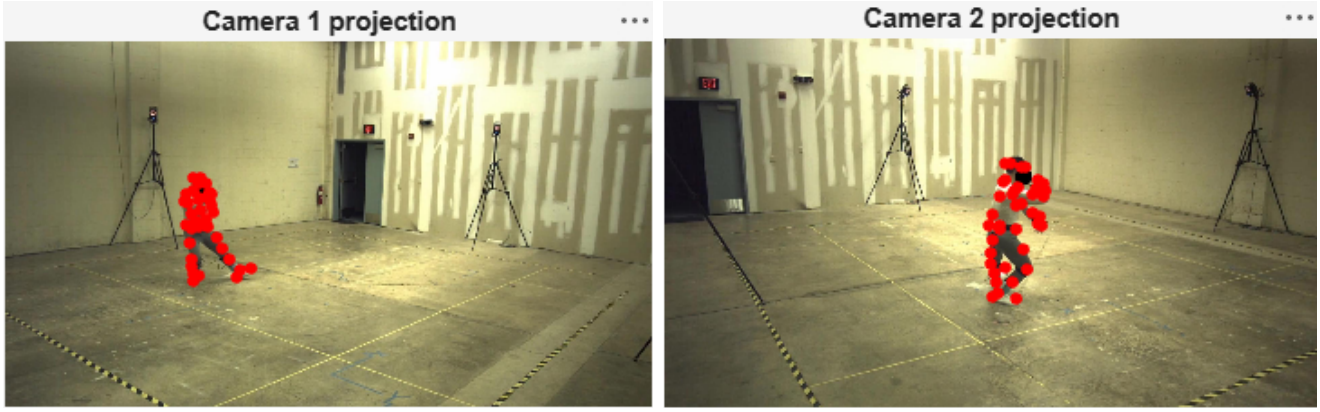
Results for Task 3.2

How we solved it:
We solved this task by first assuming that we could write a function that projected the mocap points from 3D to 2D successfully. With this assumption, we built a script that would demonstrate that the projection was successful. This meant loading in the camera parameters, the 3D points, and the images. From here, we decided that it would be nice to call a function that took all of the 3D points and one of the camera parameters, and outputted the 2D points in a 2x39 structure. So, at this point, we built a function in a

separate file called projectPoints3Dto2D with the input and output previously specified. This function first converts the 3D points to homogeneous coordinates to ensure that the rotation and translation can be applied to convert to camera coordinates. Next, the homogeneous points are multiplied by the Pmat to convert to camera coordinates. Then, the camera coordinates are turned into homogeneous pixel coordinates by being multiplied with the Kmat. Lastly, these 3x1 points are divided by their third entry to obtain the 2D pixel coordinates.

After the construction of the function, we returned to the script to call the function on the points with each camera individually. Lastly, we had to find a way to show that the projection worked correctly. To do this, we displayed the corrected images side by side in one figure. For each image, we plotted the 2D points as red dots. As seen below, the 2D projection functions as intended, as the red dots line up with the position of the person in the cameras.

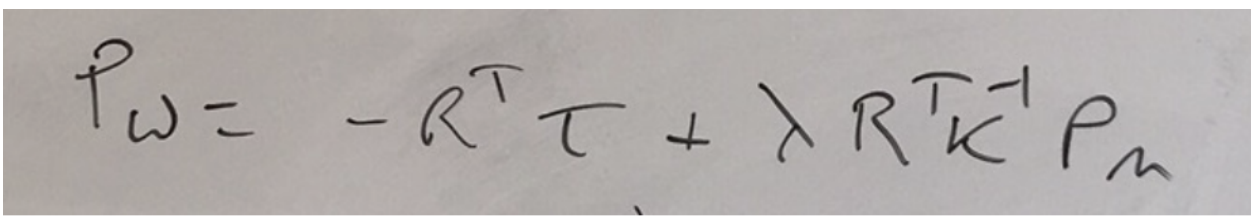
Show that 3d-to-2d projection code works correctly for both of the camera views by plotting 2D projected points on the images:



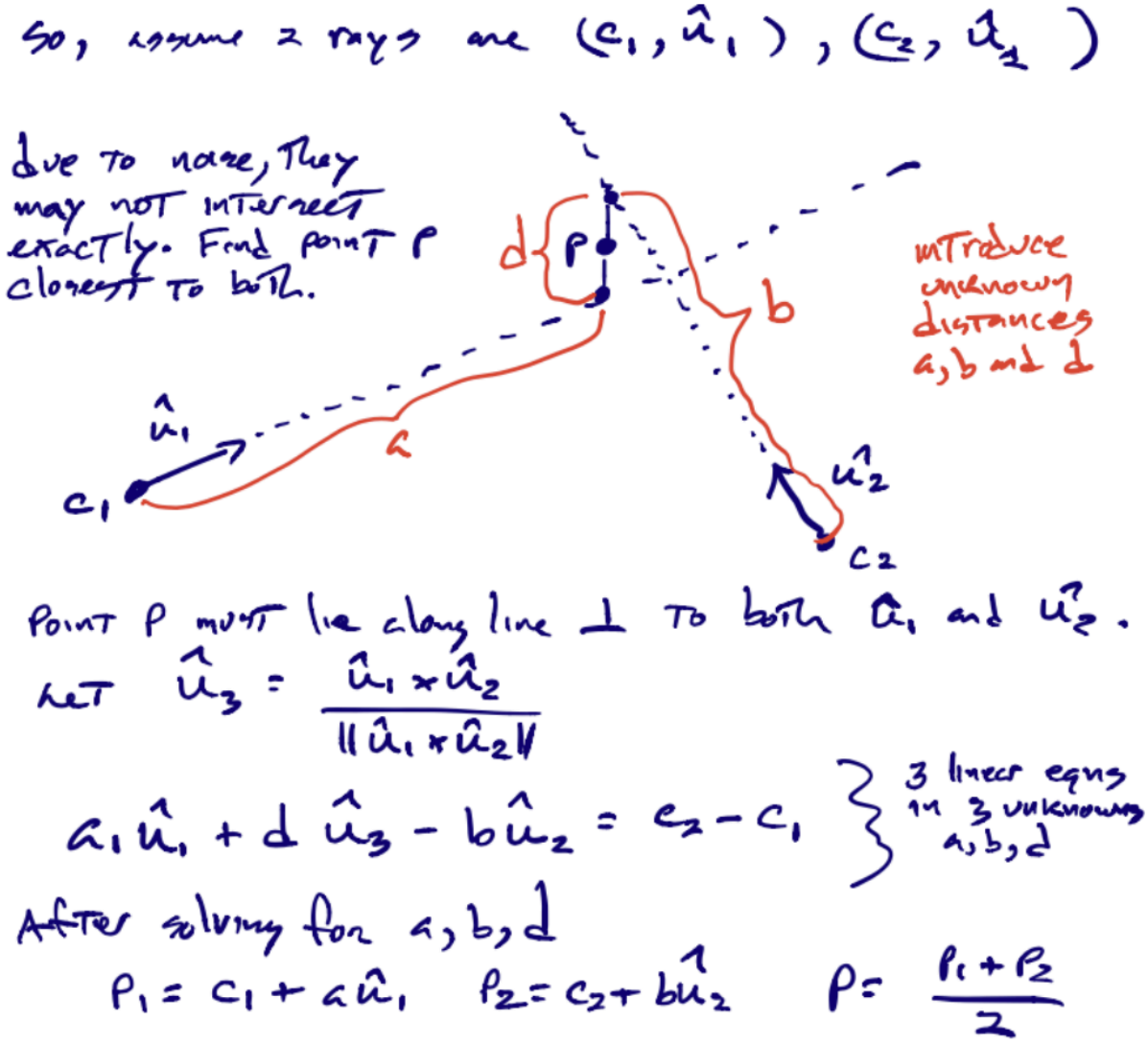
Any implementation decisions we made that were clever or unusual:
There were a few implementation decisions that may be different from other groups. The most obvious being the idea to split our implementation into two files. We felt as though this made the script much easier to read and also helped us break down the problem into smaller parts. Another decision we made was to make the projection function generalized to any number of 3D points. The main change was that we added the ones to make the homogeneous points, we added a one for each column of the points3D input rather than hard coding 39 ones. Another clever implementation decision that took a bit of research was finding a way to display the two images side by side in the same frame. We felt this made the user experience of our script much better.

Results for Task 3.3

How we solved it:
We started by projecting all 39 mocap points into both camera views using the code written for Task 3.2. This gave us two sets of corresponding 2D pixel locations. The first step in recovering the original 39 mocap 3D points was to obtain viewing rays for each set of pixel locations. To do this, we implemented viewingRays.m which uses the camera intrinsics and extrinsics to compute the camera center and the ray direction unit vector for each camera. The formulas for each are based off of this formula from the lecture slides:



The camera center is the first term in the sum and the ray direction unit vector is the second term. The next step is based off of this derivation from the lecture slides:



Here, we start by computing the cross product of the two unit vectors for each corresponding pair of rays. Once normalized to a unit vector, this gives us u_3, and we can then solve the system of equations above to compute the unknown distance a, b, and d. Once we know a, b, and d, we can solve for the point closest to the intersection of the two rays, which completes our triangulation. Lastly, to verify the code is correct, we computed the mean squared error between the original mocap points and the recovered mocap points and reported the result, which was within floating point error of 0.

To verify that your triangulation code is correct, apply it to all of the 39 mocap points that you projected and compare how close your set of reconstructed 3D points come to the original set of 3D points you started with:

As previously stated, we computed the mean squared error between the original 39 mocap points and the reconstructed mocap points and reported the result. Below is a screenshot printing the mean squared error

of our computation to the screen:

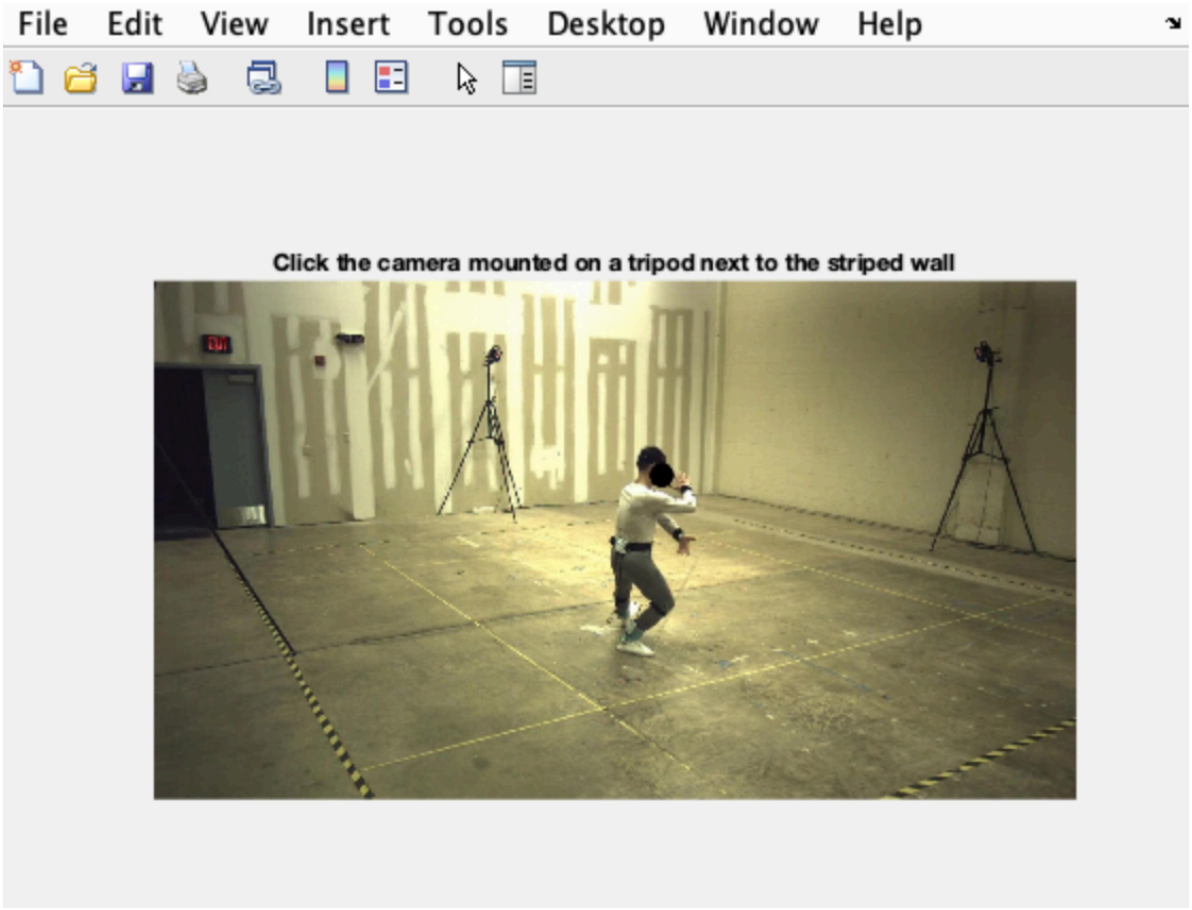
```
>> task3_3
Mean squared error: 2.86214e-24
```

The result is about 2.86*10^-24, which is extremely close to 0, so we can conclude that our triangulation code is correct.

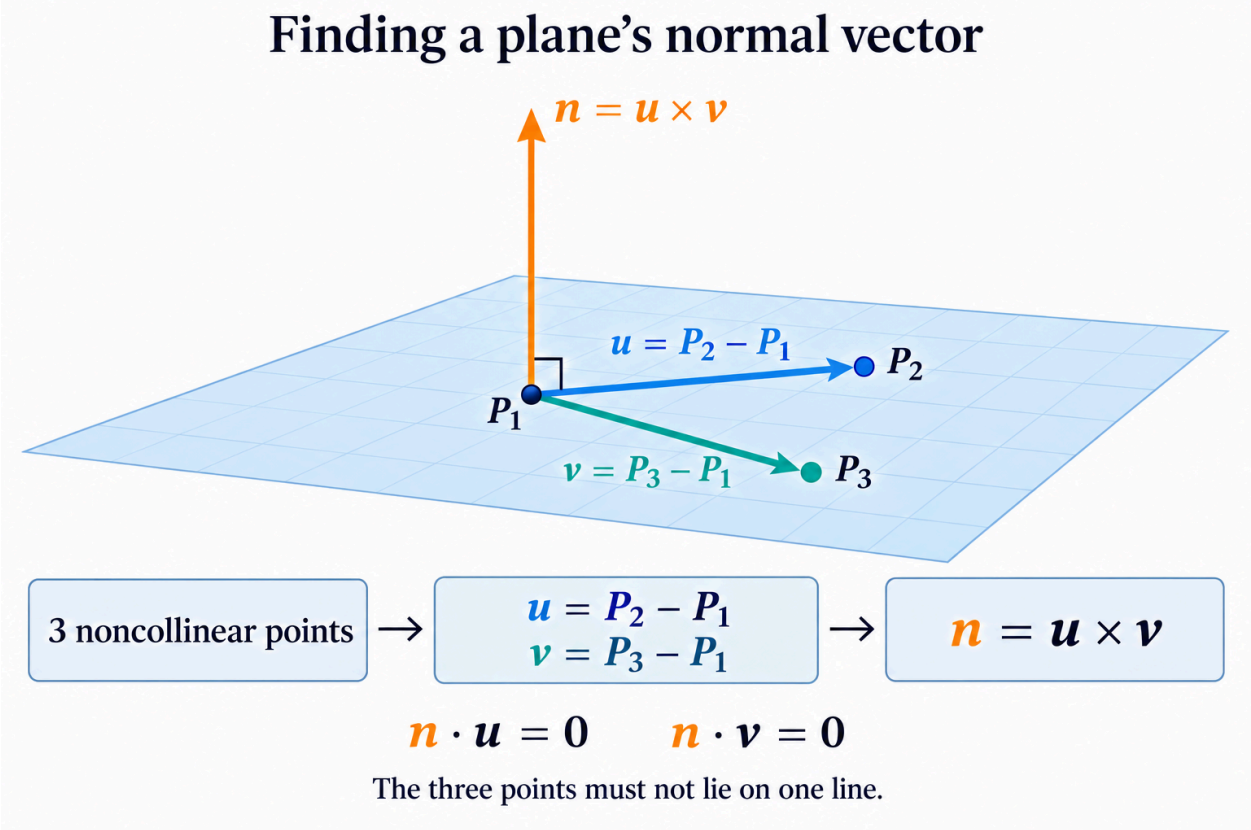
Any implementation decisions we made that were clever or unusual:
As mentioned under a previous section, logic was organized into reusable functions to make it easier to use later on. The triangulation code was useful for the next task, and abstracting away this logic avoided repetitive code. Another implementation detail that may have been different from other teams is that we used MATLAB's backslash operator to solve systems of equations. This was helpful for solving for the unknown distances a, b, and d without having to solve for each analytically and code explicit formulas. It is also MATLAB's preferred way to solve this kind of problem, and we would argue it is also more readable.

Results for Task 3.4

How we solved it:
The implementation involved 3 main parts: prompting the user to select points within the image, triangulating the corresponding points, and if necessary fitting a plane and reporting the results. We were able to follow this rough outline for all 5 components of this task. The code for prompting the user to select points was abstracted into a routine called getPoints.m, which displays each image, provides a meaningful prompt to the user (e.g. select 3 floor points), and extracts the pixel coordinates of each of the points they selected. The screenshot below is an example of what the user could be prompted with:



The code for triangulation was the same as Task 3.3, so we just reused the same routine we had implemented for that task. The code for fitting a plane involved creating the fitPlane.m routine. This function takes in a set of 3 vectors that are 3 dimensional, finds the two vectors that lie within the plane, computes the cross product of these vectors to find the normal vector defining the plane, and computes the final parameter d by solving for it directly with one of the points. The image below provides a visualization of the technique:



Once this was completed, the only remaining step was to report the results, which was straightforward and achieved by printing to the screen. For code specific to each question, refer to the sections below.

Measure the 3D locations of at least 3 points on the floor and fit a 3D plane aX+bY+cZ+d = 0 to them. Verify that your computed floor plane is (roughly) the plane Z=0.

For verifying the equation of the floor, we prompt the user to select 3 points on the floor for each camera. Each point selected should be the same point within the world for both cameras. Also, ideally the points should be spread out, to minimize the error accumulated by hand selecting them. Once that is completed, the code triangulates the 3 points, and fits a plane. The equation of the plane for the floor is reported below.

```
task3_4
The equation of the floor is 0.0081x + 0.0004y + 1.0000z+ 0.20 = 0
Floor normal is tilted 0.46 degrees from the Z axis
```

We also added code to compute the arccosine between the standard z axis basis vector and the normal vector of the plane for the floor to quantify the error we have seen. The reported error is 0.46 degrees, which can be attributed to eyeballing the points when the user selects them.

Measure the 3D locations of at least 3 points on the wall that has white vertical stripes painted on it and fit a plane. What, approximately, is the equation of the wall plane?
For finding the equation of the wall plane, we follow the same approach as for the floor, prompting the user to select 3 points from each image, triangulate to obtain world coordinates, and fit a plane. Below is a screenshot reporting the equation of the plane for the wall.

```
The equation of the wall is −0.0035x + −1.0000y + −0.0065z + −5489.25 = 0
```

From this equation, we can infer that the striped wall has a normal vector in line with the negative y axis in world coordinates. This means the wall is in line with the x axis in world coordinates, but shifted some amount from the origin.

How tall is the doorway?
To find the height of the doorway, we start by prompting the user to select a point on the top of the doorframe for each camera. From here, we can triangulate the results to obtain the 3D vector of that point in world coordinates. Under the assumption that the floor is at Z=0 in the world, the height of the doorway is exactly the z coordinate in the world frame, so we simply print out that result to the screen.

```
Doorway height: 2134.95 mm
```

From this, we can conclude that the doorway is approximately 2134 mm tall, or 2.134 m.

How tall is the person (or more precisely, how high is the top of their head at this moment)?
To find the height of the person, we follow the same exact approach as for the doorway, except now the user selects the top of the person's head. The result is shown below.

```
Person height: 1457.54 mm
```

From this, we can conclude that the person's head is approximately 1457 mm above the ground, or 1.457 m. We can not infer the person's height exactly, since they are not standing up straight.

There is a camera mounted on a tall tripod over near the striped wall; what is the 3D location of the center of that camera (roughly)?
To find the location of the center of the camera near the striped wall, we start by prompting the user to select the center of the tripod camera in both images. From here, we can triangulate the results to obtain the 3D vector of that point in world coordinates. Below, we report the results of that triangulation which gives the tripod camera center's position in the world frame.

```
Tripod camera center: [−60.1, −4946.0, 2402.5] mm
```

From this, we can infer that the center of the tripod camera next to the striped wall is located at (-60, -4946, 2402) in mm, or (-0.060, -4.946, 2.402) in m.

Any implementation decisions we made that were clever or unusual:
We were able to reuse the triangulation routine written in Task 3.3 completely unchanged, which saved time. We created a reusable function for getPoints.m, which displays the images and prompts the user to select some number of points. This saves a lot of repetitive code, especially since the code for displaying

images in MATLAB is very boilerplate to begin with. To verify that the floor is roughly the plane Z=0, we compared the computed normal vector with the standard z-axis basis vector using arccosine to quantify the angular error of this measurement.

Results for Task 3.5

How we solved it:
The task is to compute the 3x3 fundamental matrix F between the two views using only the known camera calibration, not point matches. The calibration tells us how each camera relates to the world coordinate system. However, F depends only on how the two cameras relate to each other. Most of the work is therefore deriving camera 2's rotation and position relative to camera 1. With those, we build the essential matrix E = R S and convert it to F with the K matrices.

We then used the fundamental matrix on some points in images 1 and 2 to draw epipolar lines on the opposite image. This was done as a sanity check to make sure that our E and F calculations were correct.

How do you use R1 and R2 to represent camera 2 axes with respect to the camera 1 coordinate system?
From the definition of R1, we know that it converts world coordinates to camera 1 coordinates rotation-wise. Taking the inverse of a matrix reverses its operation so we know that inv(R1) converts camera 1 coordinates to world coordinates. However, since R1 is a rotation matrix, taking the inverse is the same as the transpose so we can instead work with R1'. Since rotation matrices work to the right of the given matrix, we know that we can do R2 * R1' so that the operation works from right to left as camera 1 -> world then world -> camera 2. When we combine these operations, we get R which converts from the camera 1 coordinate system to the camera 2 coordinate system.

How do you compute the position of camera 2 with respect to the coordinate system of camera 1?
From our notes, we know that this question is asking how to compute the T vector. We can first get the arrow from camera 1 to camera 2 in world coordinates by subtracting the position of camera 1 from the position of camera 2 (C2 - C1). We can then convert this world vector into a camera 1 coordinate system vector by applying R1 to it in the form T = R1 * (C2 - C1).

Computing E and F
In order to calculate the essential matrix E, we must use the equation E = R * S. We were able to calculate R above and can get the derivation for S from our notes:

$$\text{Where S=} \begin{bmatrix} 0 & -T_z & T_y \\ T_z & 0 & -T_x \\ -T_y & T_x & 0 \end{bmatrix}$$

Since S is made up of T values, we can form the matrix from just our T calculation from above. From this, we can now calculate E = R * S. It is pretty simple to then calculate F from E as it is just the

operation F = inv(cam2.Kmat)' * E * inv(cam1.Kmat) as we saw from the notes:

$$F = K_r^{-T} R S K_l^{-1}$$

From running our code, we can get the values of R, T, E, and F as:

```
>> task3_5
Rotation of camera 2 relative to camera 1:
    0.3003    0.1312   -0.9448
   -0.2183    0.9737    0.0659
    0.9285    0.1864    0.3211

Location of camera 2 in camera 1 coordinates (mm): [-6790.6, -1030.9, 5655.5]
Essential matrix E:
   1.0e+03 *

   -0.2317    4.7169    0.5816
    5.5745    0.7871    6.8368
    1.3853   -7.4316    0.3087

Fundamental matrix F:
 -6.318271e-08   1.286088e-06  -3.971133e-04
  1.519919e-06   2.146099e-07   1.320900e-03
 -1.973886e-04  -4.461323e-03   9.999891e-01
```

Calculating and Plotting Epipolar Lines

We can check our work by plotting epipolar lines on a given image using points from the other image. First, we took the 3D points given to us and converted them into 2D coordinates for both images. As we only needed to look at a few points and epipolar lines, we took every 5th point from the two images, homogenized them, and then calculated the epipolar lines using the equation p2' * F * p1 = 0 which is shown in the notes for the film coordinates E version:

Longuet-Higgins equation

$$\underset{\text{1x3}}{p_r^T} \; \underset{\text{3x3}}{E} \; \underset{\text{3x1}}{p_l} = \underset{\text{1x1}}{0}$$

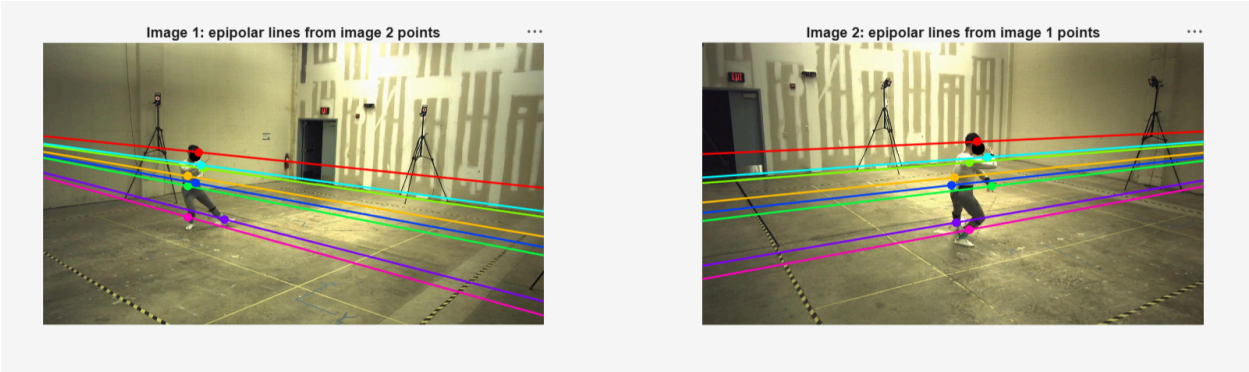
(point in image2 for p_r; point in image1 for p_l)

Epipolar lines:

$$\underset{\text{3x1}}{\begin{bmatrix} a \\ b \\ c \end{bmatrix}} = \underset{\text{3x3}}{E} \; \underset{\text{3x1}}{p_l} \qquad [a \;\; b \;\; c] = \underset{\text{1x3}}{p_r^T} \; \underset{\text{3x3}}{E}$$

(line in image2 for $[a\ b\ c]^T$; point in image1 for p_l; line in image1 for $[a\ b\ c]$; point in image2 for p_r)

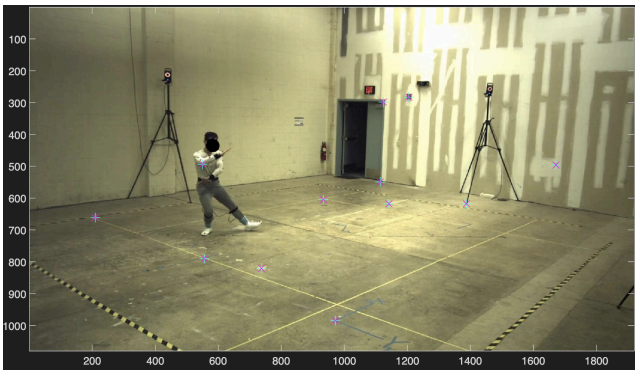
We then plotted these points and epipolar lines on the two images:

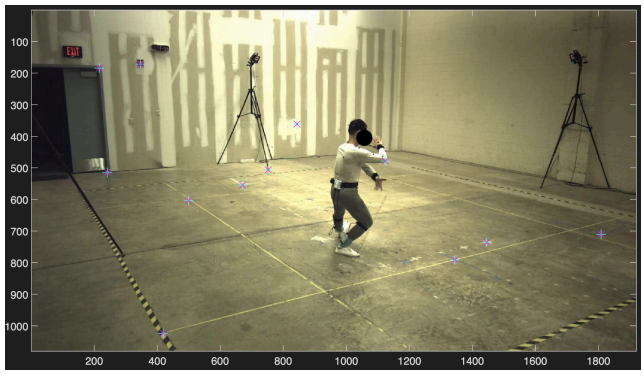


Results for Task 3.6

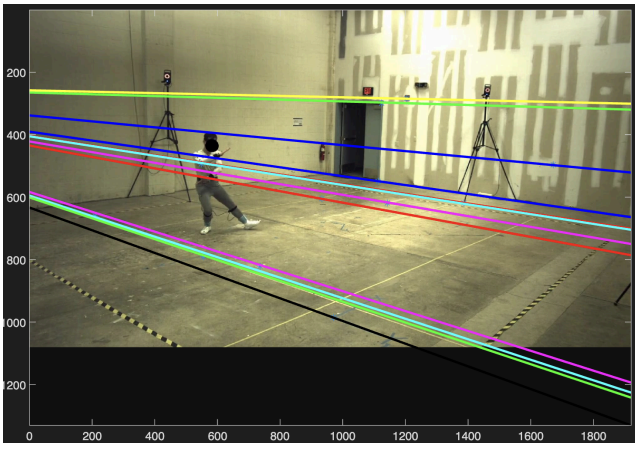
How we solved it:
To compute the fundamental matrix using the eight point algorithm, we first selected 12 unique points in the provided image, ensuring that such points corresponded to the same physical locations and varied in depth. For this particular image, we used the following sequence of points: Bottom-right corner of the doorway, top right corner of the doorway, the center tripod leg of the camera located to the right of the door, the intersection of striped and solid tape behind the individual, the plus sign farthest from the individual in the image, the fire alarm, the corner of blue axis labels nearest to the first image, the black mark in the wall next to the tripod, the blue-yellow line intersection nearest to the subject, the solid yellow line intersection nearest to the door, the subject's right elbow, and the corner formed by the white lines nearest to the door.

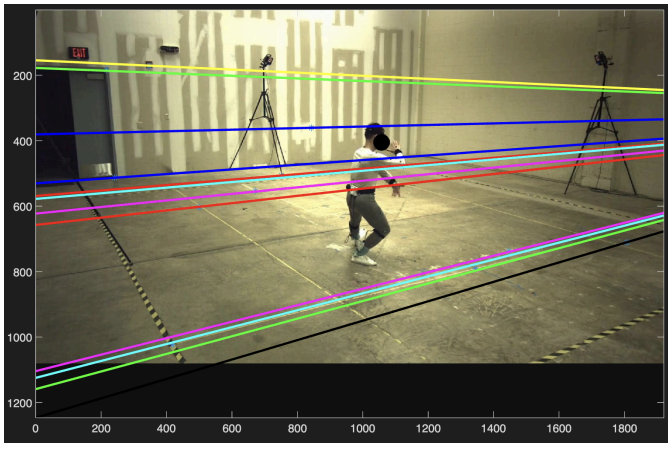
After selecting the corresponding points in both images, we executed the eight-point algorithm to estimate the fundamental matrix and generate epipolar lines in both camera views.





Plotting these points in both images and running the algorithm produced the following epipolar lines:





The resulting epipolar geometry appears reasonable, where the estimated epipolar lines generally follow the expected geometric relationships between the two camera views. That said, some discrepancies are visible, which can be attributed to inaccuracies in manually selecting corresponding points and the limited number of observations (12) used to estimate the fundamental matrix. In 3.7 we will quantitatively evaluate the accuracy of this estimate.

Output:
After selecting the necessary points, the algorithm produced the following fundamental matrix:

```
Fundamental Matrix (F):
      0.00000006     -0.00000091      0.00028231
     -0.00000094      0.00000012     -0.00105816
      0.00008205      0.00284809     -0.57406025
```

Results for Task 3.7

How we solved it:

To compute the symmetric epipolar distance (SED) for both the calibrated and eight-point algorithm fundamental matrices, we first loaded the camera parameters and the 39 provided 3D points. We then projected the 3D points into the 2D space where we converted them into homogeneous coordinates so that they can be used in 3 dimensional matrix operations.

Next we computed the calibrated fundamental matrix using our implementation from Task 3.5 and loaded the estimated fundamental matrix generated in Task 3.6. For each of the 39 corresponding point pairs, we computed the associated epipolar lines in both images using the following equations:

$$l_r = F p_l \quad l_l = F^T p_r$$

Using the epipolar lines and the 2D points from both images, we compute the normalized perpendicular geometric distances from each point to its corresponding epipolar line. For each fundamental matrix, we found the sum of the squared distances across all 39 point correspondences and both camera views. Finally, we divided the accumulated error by 78, which represents the total number of squared distances to obtain the mean squared geometric distance in pixels.

```
Calibrated F SED:   2.217877e-26 pixels^2
Eight-Point F SED:  13.362753 pixels^2
```

$\sqrt{SED} = \sqrt{13.36} \approx 3.655$ *pixels*

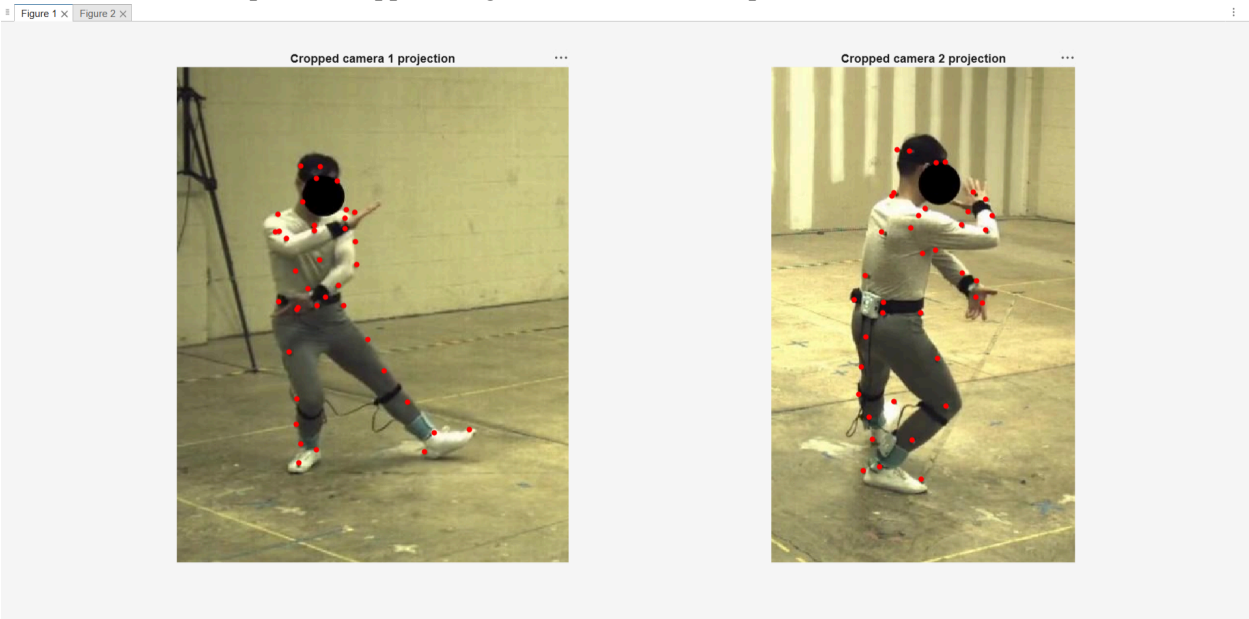
The calibrated matrix's near-zero error is expected as the point correspondences were generated using the same camera calibration parameters that were utilized to derive the fundamental matrix.

The squared error produced by the eight-point algorithm is larger (13.36 pixels^2), meaning that the points typically deviate from the epipolar lines by ~3.655 pixels. This may be attributed to inconsistent marking

of shared patches across two very different views and restrictions associated with estimating the fundamental matrix from points that were hand-selected. That said, from epipolar lines constructed in 3.6, we can see that the algorithm is generally effective in mapping the geometric relationship between the two camera views in the absence of camera calibration parameters, though its larger SED indicates lower accuracy than the calibrated matrix.

Results for Extra Credit 1

How we solved it:
We first got the 2D points for the motion capture dots given the 3D locations like usual. We then used these points to create a bounding box for both images. This bounding box was calculated using a padding distance from the minimum and maximum for both the column and row pixels of the 2D motion capture points. Next, the cropView function crops each image to its new rectangle and updates the cameras with their new properties. The K matrices have to be updated with the new principal points for the two views. Once this is done, new image files are created for the cropped views. We can then project the 3D coordinates of the motion capture dots into these cropped views using our old projectPoints3Dto2D function. We then output the cropped images with the motion capture dots identified.



After this, we move onto computing the new F matrix so that we can calculate and display the epipolar lines for both cropped images. This is as simple as passing the new camera properties into our computeFundamental function. The epipolar lines for the 2D cropped points are then calculated and the error from the corresponding points is outputted as it should be close to 0. We then also compute the old F matrix for the uncropped images to show how cropping makes the old F invalid for this new cropped view.

```
>> extraCredit1
Crop rectangle view 1 [col, row, width, height]: [433, 304, 401, 506]
Crop rectangle view 2 [col, row, width, height]: [853, 264, 370, 602]
Cropped Kmat (camera 1):
   1.0e+03 *

    1.5607         0    0.5355
         0    1.5607    0.2457
         0         0    0.0010

Cropped Kmat (camera 2):
   1.0e+03 *

    1.5279         0    0.1179
         0    1.5279    0.2869
         0         0    0.0010

Max difference from shifted full-image projection (pixels): view 1 2.27374e-13, view 2 3.69482e-13
Fundamental matrix F for the cropped views:
 -8.943685e-07   1.820492e-05  -4.915222e-04
  2.151487e-05   3.037862e-06   2.891260e-02
  2.102320e-03  -4.684169e-02   9.984815e-01
Mean distance of cropped points to their epipolar lines (pixels):
  cropped F:  2.35026e-13
  original F: 149.646
>>
```

We finish off the extra credit with picking a few points in the cropped view and calculating their respective epipolar lines. This is done so we can display the outputs.

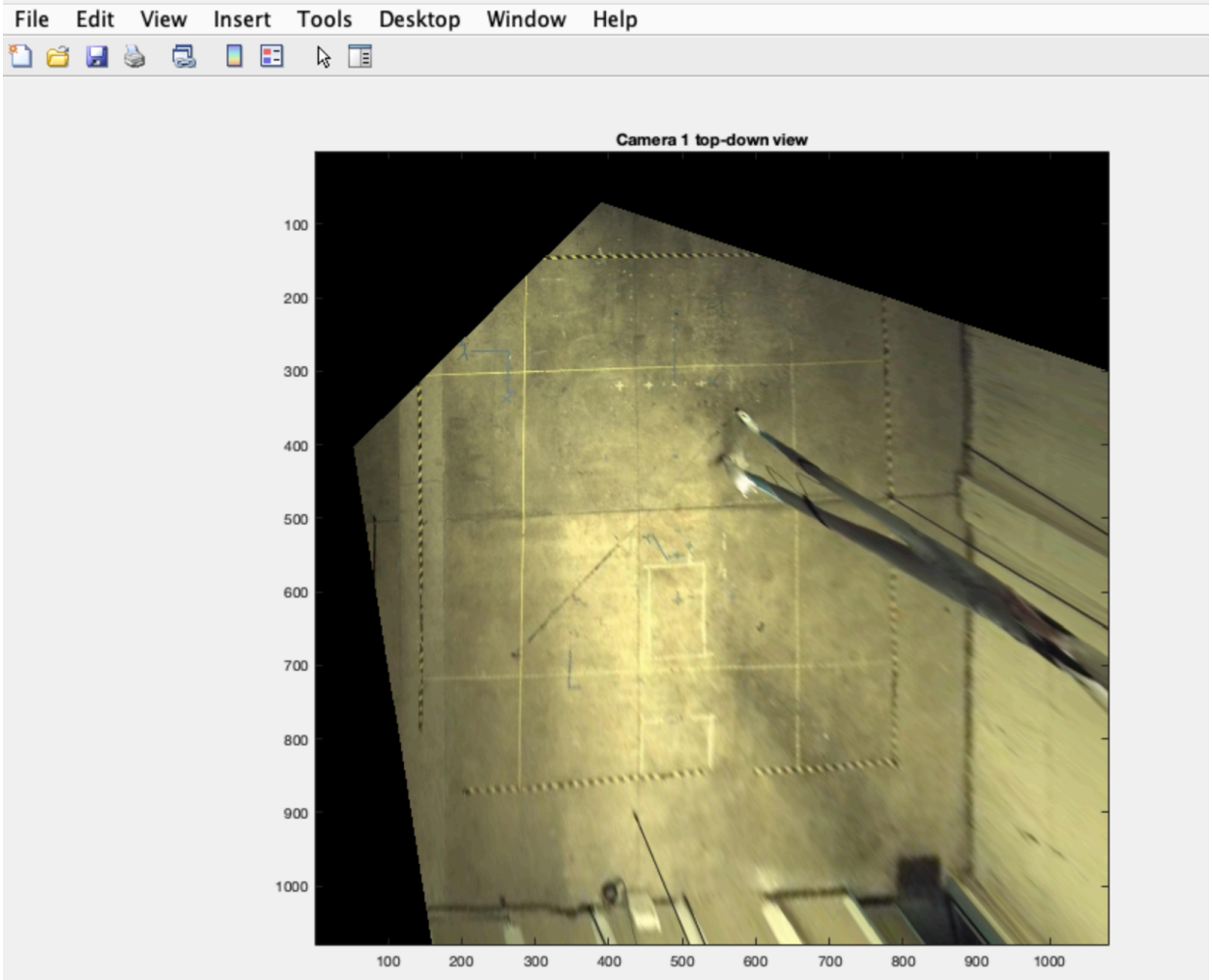


Results for Extra Credit 2

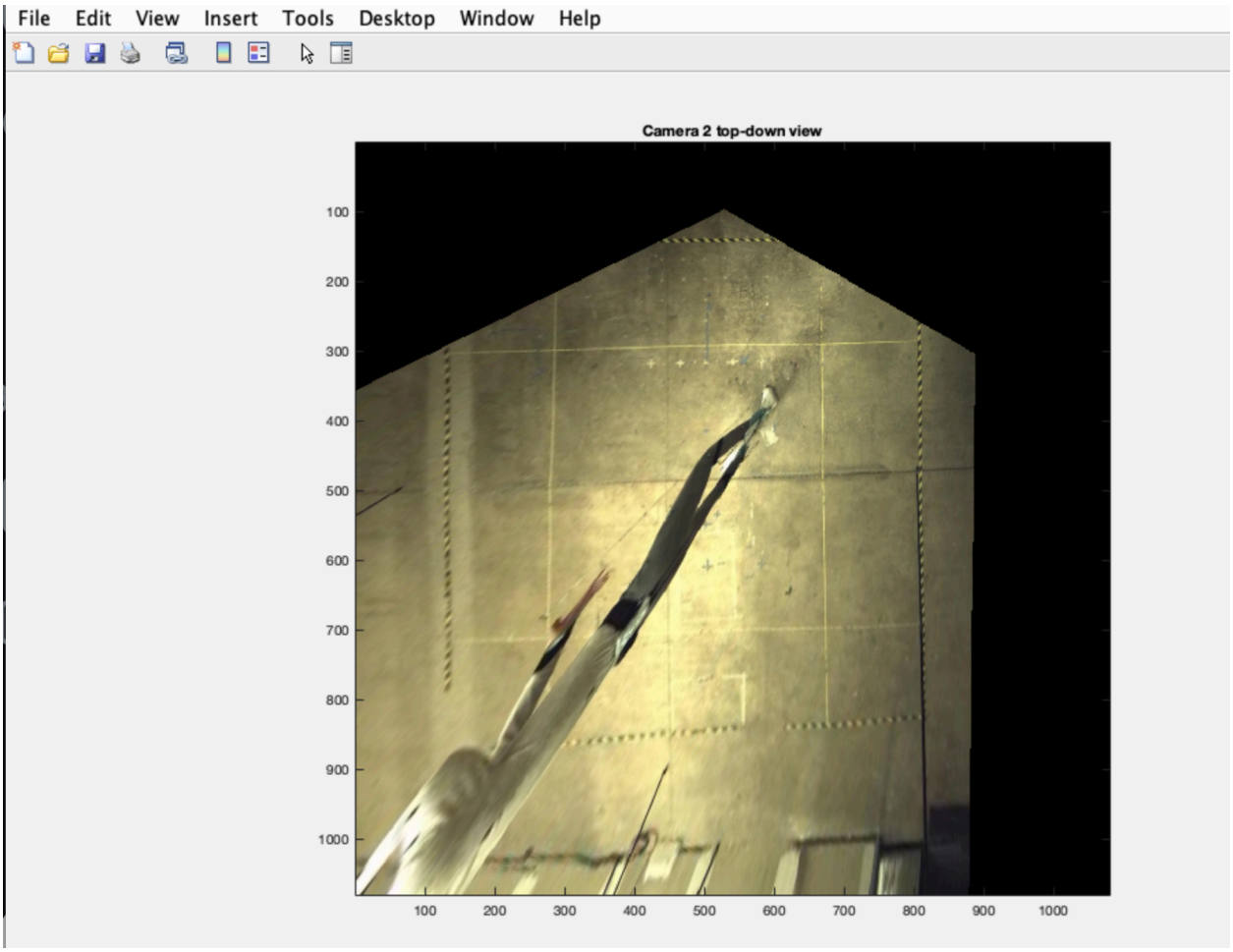
How we solved it:

We modified the provided planewarpdemo.m script from the course to use known camera extrinsics and intrinsics instead of asking for user input of corresponding points. We then generated a top down view for each of the two cameras using the same floor region. We also modified the demo code to increase the output image resolution to match the resolution of the input images.

Explain how you are generating your top-down view.
First, we combine each of the camera's intrinsic and extrinsic matrices to obtain the world to image matrix, which we call P. Since we know that the floor is the plane Z=0, we can drop the third column from this matrix since it contributes nothing. This new matrix is the homography which maps the floor coordinates into the source image coordinates. We then construct the output image representing a fixed region of the floor, which is roughly defined by the coordinates of several hardcoded points at the corners of the floor in world coordinates. Using these bounds, we compute the matrix we call S that converts output pixels in world coordinates using translation and scaling along both axes. When we multiply the S matrix by the homography, we obtain the complete transformation going from the location in the source image to the location in the output image. We interpolate the colors of the image since some resolution is lost through this projection. The results for camera 1 are shown below:



And the results for camera 2 are shown below:



In both images, you can see we have reproduced a top down view that is accurate up to a rotation, translation, and isotropic scale.

Also, with regard to the resulting output image, what things look accurate and what things look weird?
The floor markings look accurate. Straight, parallel lines remain straight and parallel. Angles and relative distances on the floor are also preserved.

Anything that extends above the floor is inaccurate. This would include the person, walls, and camera tripods. Since the transformation is assuming that every point lies on the Z=0 plane, the top down views of objects not on that plane appear incorrect. Some areas of the image also appear blurry since the original camera views are not top down, and we need to use interpolation to fill in areas of the image that are undersampled (with respect to the input image resolution) when we project to the top down view. Black regions correspond to locations that are not visible from the source image, and we did not attempt to fill these in.

Could this kind of view be useful for analyzing anything about the performance of a person as they move around in the room?
This top down view could potentially be useful for analyzing the movement of the person around the room. You can use some kind of model to track the feet of the person, and from the known world coordinate transformation, we can compute the ground distance traveled across multiple frames of a

video. You can also derive velocity information from this as well. In general, it would only be useful for measurements close to the ground (like the person's feet) since elevated surfaces are not represented properly in this view.

Any implementation decisions we made that were clever or unusual:
We derived the homographies directly from the intrinsic and extrinsic parameters, rather than estimating them from manually selected points as in the original demo code. We also used hard coded bounds in world coordinates. These bounds were obtained by selecting corresponding floor corners in both images and triangulating for world coordinates using the code from task 3.4. These bounds were then rounded off and used to control the crop and scale of the output image.

Team Contributions

Responsibilities were divided among team members, with each person taking on 2-3 tasks. We believe everyone's contributions were approximately equal. The table below summarizes who worked on each task.

Assignment (Code + Write-Up)	**Contributor**
Task 3.1	Ben Rishel
Task 3.2	Ben Rishel
Task 3.3	Cole Kvasnak
Task 3.4	Cole Kvasnak
Task 3.5	Gavin Sweeney
Task 3.6	Kyle Nicarry
Task 3.7	Kyle Nicarry
Extra Credit 1	Gavin Sweeney
Extra Credit 2	Cole Kvasnak